



Python Collections

Lists and Tuples, Indexing and Slicing,
String, Dictionaries and Sets, Mutability Concepts

Python Collections

There are four collection data types in the Python programming language:

- List is a collection which is ordered and changeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- Set is a collection which is unordered and unindexed. No duplicate members.
- Dictionary is a collection which is unordered, changeable and indexed. No duplicate members.

Lists

- A list is a collection which is ordered and changeable.
- The list is a datatype available in Python which can be written as a list of comma separated values (items) between square brackets.
- Items in a list need not be of the same type.

```
list1 = ['physics', 'chemistry', 1997, 2000]  
list2 = [1, 2, 3, 4, 5 ]  
list3 = ["a", "b", "c", "d"]  
print(list3)
```

Access Items

- To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.
- Negative indexing means beginning from the end, -1 refers to the last item, -2 refers to the second last item etc. eg List[-1]

```
print(list2[0])  
print(list1[0:3])  
print(list3[-1])
```

Updating list

- You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and
- To add an item to the end of the list, use the `append()` method.
- To add an item at the specified index, use the `insert()` method.

```
list1 = ['physics', 'chemistry', 1997, 2000]  
print(list1[2])  
# Updating the list  
list1[2]="Math"  
print(list1)
```

What is the output of the following code?

```
l1=[12,34,56,78]  
print(l1[2])
```

A. 12

B. 34

C. 56

D. 78

What is the output of the following code?

```
l1=[12,34,56,78]  
print(l1[2])
```

A. 12

B. 34

C. 56

D. 78

What is the output of the following code?

```
l1=[1,2,0,-2,-1]  
print(l1[-1])
```

A. Error

B. 2

C. -1

D. 1

What is the output of the following code?

```
l1=[1,2,0,-2,-1]  
print(l1[-1])
```

A. Error

B. 2

C. -1

D. 1

Slicing

- Slicing in Python is a technique used to extract a portion (a "slice") of a sequence—such as a string, list, or tuple—by specifying a range of indices.
- The standard syntax for slicing uses square brackets with up to three parameters separated by colons: `sequence[start:stop:step]`
- *start*: The index where the slice begins (inclusive). If omitted, it defaults to 0.
- *stop*: The index where the slice ends (exclusive). The slice goes up to, but does not include, this index. If omitted, it defaults to the length of the sequence.
- *step*: (Optional) The increment between each index. It defaults to 1.

Slicing

- Negative Indexing
- Python allows you to count from the end of a sequence using negative numbers. The last element is -1, the second to last is -2, and so on.
- Extract last 3 characters: `text[-3:]`.
- Everything except the last character: `text[:-1]`.

- Using the Step Parameter
- The step value determines how many elements to skip.
- Every second element: `text[::2]`
- Reversing a sequence: A negative step of -1 reverses the order.

Dictionary

- A dictionary (or dict) is a built-in data structure used to store data in key-value pairs.
- Mutable: You can add, remove, or change items after the dictionary is created.
- Unique Keys: Keys must be unique; if you add a duplicate key, the new value will overwrite the old one.
- Key Types: Keys must be immutable (unchangeable) types like strings, numbers, or tuples.
- Fast Lookups: Dictionaries use hash tables internally, allowing for near-instant data retrieval even with millions of items.

Dictionary

```
# Create empty dictionary
d2=dict()
print(d2)
#Create a dictionary with key value pair
d1={'a':12,'b':13,'c':56}
print(d1)
#print keys only
print(d1.keys())
#print values only
print(d1.values())
# Add or update
d1['new']=100
print(d1)
# print key value pair
print(d1.items())
# delete one key value pair
del d1['c']
print(d1)
```

```
{
'a': 12, 'b': 13, 'c': 56}
dict_keys(['a', 'b', 'c'])
dict_values([12, 13, 56])
{'a': 12, 'b': 13, 'c': 56, 'new': 100}
dict_items([('a', 12), ('b', 13), ('c', 56), ('new', 100)])
{'a': 12, 'b': 13, 'new': 100}
```

What is the output of the following code?

```
l1={'apple':12,'banana':24,'orange':10}  
print(l1.values())
```

A. [12, 24, 10]

B. {'apple': 12, 'banana': 24, 'orange': 10}

C. ['apple', 'banana', 'orange']

D. 12, 24, 10

What is the output of the following code?

```
l1={'apple':12,'banana':24,'orange':10}  
print(l1.values())
```

A. [12, 24, 10]

B. {'apple': 12, 'banana': 24, 'orange': 10}

C. ['apple', 'banana', 'orange']

D. 12, 24, 10

Tuples

A tuple is a built-in data type used to store a collection of items in a specific order. The most defining feature of a tuple is that it is immutable, meaning once you create it, you cannot change, add, or remove its elements.

- Ordered: Items have a defined order that will not change.
- Immutable: You cannot modify elements after creation. If you need to change data, you must create a new tuple.
- Allow Duplicates: Since they are indexed, they can store multiple items with the same value.
- Heterogeneous: A single tuple can hold elements of different data types

Tuple

```
>>> t=(12,"abc",23.45,[12,34],10)
>>> t
(12, 'abc', 23.45, [12, 34], 10)
>>> print(t[3])
[12, 34]
```

What is the output of the following code?

```
t=(1,2,3,4)
t[0]=0
print(t)
```

A. (1,2,3,4)

B. (0,2,3,4)

C. (0,1,2,3,4)

D. Error

What is the output of the following code?

```
t=(1,2,3,4)
t[0]=0
print(t)
```

A. (1,2,3,4)

B. (0,2,3,4)

C. (0,1,2,3,4)

D. Error

String

- A string is a sequence of characters enclosed in quotes.
- String Properties
 - Ordered
 - Immutable
 - Indexed

```
>>> s1="MCA"  
>>> print(s1)  
MCA  
>>> s2='BCA'  
>>> print(s2)  
BCA
```

Set

- A set is an unordered collection of unique, hashable items
- Unique Elements: Duplicate values are automatically removed upon creation or when adding new items.
- Mutable: You can add or remove elements after the set is created, though the elements themselves must be of an immutable data type

Set

- `my_set = {"apple", "banana", "cherry"}.`
- Empty Set: You must use `set()`. Using `{}` creates an empty dictionary instead.
- Constructor: Useful for converting other iterables like lists into sets to remove duplicates: `unique_items = set([1, 2, 2, 3])`

